

Dokumentation

“Webanwendung Sparo”

Elias Schultze

ON24-3, November 2025

Inhaltsverzeichnis

1. Thema des Projekts	5
1.1. Was soll die Anwendung können?.....	5
1.2. Warum habe ich mich für dieses Thema entschieden?	5
2. Ausgangssituation	6
2.1. Persönliche Ausgangssituation	6
2.2. Bisherige Erfahrungen in der Webentwicklung	6
3. Vorgehen	7
3.1. Planung	7
3.2. Vorgehensweise	8
3.3. Wissenserwerb	8
3.4. Wissensweitergabe	8
4. Anforderungsliste	9
5. Konzeption	12
5.1. Technologie-/Werkzeugauswahl.....	12
5.1.1. Backend	12
5.1.2. Frontend.....	13
5.2. Entwurf.....	14
5.2.1. Client-Server-Architektur mit REST-API	14
5.2.2. Komponenten und Zusammenwirken	14
5.2.3. Entscheidungen	15
5.2.4. UI-Entwurf	16
5.2.5. Datenmodell.....	16

5.2.6.	Persistenzüberlegungen	17
5.2.7.	Interaktionsideen zwischen Client und Server	18
5.2.8.	Datei-Übersicht.....	21
6.	Ergebnis des Projekts	24
6.1.	Funktionalität	24
6.2.	Screenshots	26
7.	Reflexion.....	31
7.1.	Herausforderungen	31
7.2.	Unterstützung.....	33
7.3.	Lernerfolge / Fazit.....	34
7.3.1.	Was habe ich gelernt?.....	34
7.3.2.	Wie kann ich das Wissen zukünftig einsetzen?	35
7.3.3.	Was lief gut?	36
7.3.4.	Was würde ich verbessern?	36
7.3.5.	Was kann man nun mit der Anwendung machen?	36
7.3.6.	Wie kann sie weiterentwickelt werden?.....	37
7.4.	KI-Einsatz.....	37
7.4.1.	Verwendete KI-Systeme	37
7.4.2.	Wie wurde KI eingesetzt?	38
7.4.3.	Transparenz und Verantwortung	39
A.	Installationsanleitung	40
B.	Benutzerdokumentation.....	43
	Literaturverzeichnis.....	48

1. Thema des Projekts

1.1. Was soll die Anwendung können?

Sparo ist eine vollständig funktionsfähige Budget-Tracking-Webanwendung, die es Benutzern ermöglicht:

- **Persönliche Finanzen zu verwalten:** Einnahmen und Ausgaben zu erfassen, zu kategorisieren und zu verfolgen
- **Finanzübersicht zu erhalten:** Monatliche Salden, Einnahmen- und Ausgabenstatistiken sowie Kategorien-Analysen
- **Sparziele zu setzen und zu verfolgen:** Individuelle Sparziele mit Zielbetrag, Zeitrahmen und Fortschrittsanzeige
- **Transaktionen zu verwalten:** Vollständige CRUD-Operationen für alle Finanztransaktionen

1.2. Warum habe ich mich für dieses Thema entschieden?

Budget-Tracking ist eine alltägliche Anwendung, die viele Menschen nutzen würden. Dies macht das Projekt relevant und motivierend. Die Anwendung erfordert alle grundlegenden Datenbankoperationen (Create, Read, Update, Delete), was ideal ist, um die erlernten Konzepte praktisch anzuwenden. Das Projekt ist komplex genug, um moderne Webentwicklungsprinzipien zu demonstrieren (Client-Server-Trennung, REST-API, Authentifizierung), aber nicht so komplex, dass es unüberschaubar wird.

Die Architektur ist durchdacht, was sich durch den Raum für zukünftige Erweiterungen (z.B. wiederkehrende Transaktionen, Budget-Limits, Export-Funktionen) zeigt.

Darüber hinaus deckt das Projekt alle geforderten Aspekte ab:

- Trennung von Client und Server

- Session-Management (JWT)
- Benutzerauthentifizierung
- Asynchrone Kommunikation (REST-API)
- Zentrale Datenhaltung
- Vue.js mit Reactivity
- Sicherheitsmaßnahmen

2. Ausgangssituation

2.1. Persönliche Ausgangssituation

Vor diesem Projekt hatte ich bereits Grundkenntnisse in:

- **HTML/CSS/JavaScript:** Fortgeschrittenes Wissen für Frontend-Entwicklung
- **Vue.js:** Grundlegende Kenntnisse in Vue 3
- **Node.js/Express:** Basiswissen für Backend-Entwicklung
- **SQL/MySQL:** Grundkenntnisse in relationalen Datenbanken
- **Git:** Basiswissen für Versionskontrolle

2.2. Bisherige Erfahrungen in der Webentwicklung

- Einfache statische Websites (Onepager, Shopsysteme, CMS)
- Grundlegende Vue.js-Komponenten
- Keine Erfahrung mit:
 - Deno
 - JWT-Authentifizierung
 - Docker/Docker Compose
 - Pinia State Management

- REST-API-Design
- Datenbank-Indizierung und Performance-Optimierung

Skill-Übersicht (vor Projekt)

Skill	Level
HTML/CSS	Fortgeschritten
JavaScript	Mittel
Vue.js	Grundlagen
Node.js/Express	Grundlagen
SQL/MySQL	Grundlagen
REST-APIs	Anfänger
Docker	Anfänger
Git	Grundlagen

3. Vorgehen

3.1. Planung

- **Anforderungsanalyse:** Definition der Features (Authentifizierung, Transaktionen, Sparziele)
- **Technologie-Stack-Auswahl:** Entscheidung für Deno, Vue 3, MySQL
- **Datenmodell-Design:** ER-Diagramm für Datenbank-Schema
- **API-Design:** Definition der REST-Endpunkte

3.2. Vorgehensweise

Phase 1: Fokus liegt auf der Erstellung des Datenbankschemas und des Oak v16 Servers mit Router. Implementiert werden eine Body-Parser Middleware, Authentifizierungs-, Transaktions- und Sparziel-Endpunkte sowie eine JWT-Middleware zur Absicherung der Routen.

Phase 2: Die Phase umfasst die Initialisierung des Vue 3 Projekts, die Konfiguration des Routers und die Einrichtung des Pinia Stores. Es werden die Kern-Ansichten wie Login/Registrierung, das Dashboard, das Transaktions-Formular und der Sparziel-Tracker entwickelt.

Phase 3: In dieser Phase erfolgt die Frontend-Backend-Integration, die CORS-Konfiguration, die Implementierung einer robusten Fehlerbehandlung und der abschließende UI-Polish.

Phase 4: Die letzte Phase beinhaltet die Docker-Containerisierung und das Docker Compose Setup zur Koordination der Dienste, gefolgt von der Erstellung der Dokumentation.

3.3. Wissenserwerb

Die Lernressourcen umfassen die Offiziellen Dokumentationen (Deno, Oak v16, Vue 3, MySQL) und spezialisieren sich auf die Oak v16 Migration (insbesondere Body-Parsing via bodyReader). Zusätzlich werden YouTube-Tutorials (Vue 3 Composition API, JWT, Oak) und Stack Overflow für spezifische Lösungen genutzt. GitHub-Repositories dienen als Quelle für Best Practices, während KI-Tools (siehe Abschnitt 7.4) zur Code-Erklärung und zum Debugging eingesetzt werden.

3.4. Wissensweitergabe

- Code-Kommentare für zukünftige Wartung
- Diese Dokumentation für andere Entwickler

4. Anforderungsliste

	Anforderung	Fertigstellungsgrad	Umsetzung
1	Benutzerregistrierung	100%	Vollständig implementiert mit E-Mail-Validierung und Passwort-Hashing
2	Benutzerlogin	100%	JWT-basierte Authentifizierung mit Token-Speicherung
3	Einnahmen hinzufügen	100%	POST /api/transactions/income mit Validierung
4	Ausgaben hinzufügen	100%	POST /api/transactions/expense mit Validierung
5	Transaktionen anzeigen	100%	Dashboard zeigt monatliche Zusammenfassung
6	Transaktionen	100%	PUT

	bearbeiten		/api/transactions/ transaction/:id
7	Transaktionen löschen	100%	DELETE /api/transactions/ transaction/:id
8	Saldo-Berechnung	100%	Automatische Berechnung im Dashboard
9	Kategorien- Übersicht	100%	GROUP BY category in SQL- Query
10	Sparziele erstellen	100%	POST /api/transactions/ goal
11	Sparziele anzeigen	100%	GET /api/transactions/ goals und GET /api/transactions/ goal/:id
12	Sparziele bearbeiten	100%	PUT /api/transactions/ goal/:id
13	Sparziele löschen	100%	DELETE /api/transactions/ goal/:id

14	Sparziel-Fortschritt	100%	Fortschrittsbalke n mit Prozentanzeige
15	Monatliche Sparbeträge	100%	Automatische Berechnung basierend auf Enddatum
16	Fehlerbehandlung	100%	Try-Catch-Blöcke und Benutzer- Feedback
17	Docker- Containerisierung	100%	Dockerfile und docker- compose.yml
18	Datenbank- Indizierung	100%	Indizes für Performance- Optimierung
19	CORS-Konfiguration	100%	Whitelist für erlaubte Origins
20	JWT-Token- Expiration	100%	Tokens laufen nach 1 Tag ab
21	Transaktion löschen/bearbeiten	0%	Leider nicht geschafft

Gesamt-Fertigstellungsgrad: 98,95%

5. Konzeption

5.1. Technologie-/Werkzeugauswahl

5.1.1. Backend

Deno Runtime - Begründung: Deno bietet moderne JavaScript/TypeScript-Unterstützung ohne Node.js-Overhead. Die integrierte Sicherheit durch explizite Permissions (`--allow-net`, `--allow-read`) und native TypeScript-Unterstützung machen Deno ideal für sichere Backend-Entwicklung. Die native Unterstützung für npm-Pakete ermöglicht die Nutzung bewährter Bibliotheken wie `mysql2`, `bcryptjs` und `jsonwebtoken`.

Oak Framework v16 - Begründung: Oak ist das native Web-Framework für Deno und wurde entsprechend der Aufgabenstellung ("deno mit oak") verwendet. Oak ist speziell für Deno entwickelt und nutzt native Deno-Features wie TypeScript-Unterstützung und moderne Async/Await-Patterns. Oak ist vollständig in TypeScript geschrieben und bietet bessere Type-Safety. Oak v16 verwendet Context-basierte Middleware mit `ctx`-Objekt, das Request und Response kapselt.

MySQL 8.0 - Begründung: MySQL ist eine relationale Datenbank mit ausgezeichneter Performance und Stabilität. Für Budget-Tracking-Anwendungen ist die relationale Struktur ideal, um Transaktionen, Benutzer und Sparziele mit Foreign Keys zu verknüpfen. MySQL 8.0 bietet moderne Features wie JSON-Unterstützung und verbesserte Performance.

JWT (JSON Web Tokens) - Begründung: JWT ermöglicht stateless Authentifizierung, was für REST-APIs ideal ist. Tokens können im Frontend gespeichert werden und enthalten alle notwendigen Benutzerinformationen. Die Verwendung von `jsonwebtoken` ist sicher.

bcryptjs - Begründung: Passwörter müssen gehasht werden, da sie niemals im Klartext gespeichert werden sollen. `bcryptjs` ist eine bewährte Bibliothek für Passwort-Hashing.

5.1.2. Frontend

Vue.js 3 - Begründung: Vue.js bietet eine reaktive, komponentenbasierte Architektur mit geringer Lernkurve. Die Composition API ermöglicht wiederverwendbaren, testbaren Code.

Vue Router - Begründung: Für Single-Page-Applications (SPAs) ist ein Router unerlässlich. Vue Router bietet Navigation Guards für Authentifizierung, dynamische Routen und History-Management.

Axios - Begründung: Axios ist eine moderne HTTP-Client-Bibliothek mit Interceptors, automatischem JSON-Parsing und besserer Fehlerbehandlung als fetch. Ideal für REST-API-Kommunikation.

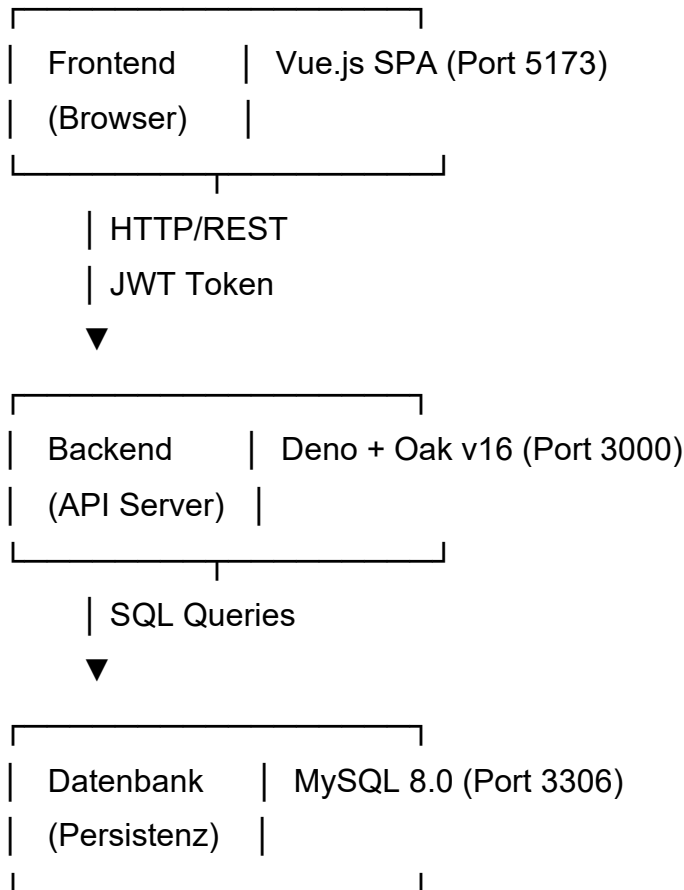
Vite - Begründung: Vite ist ein moderner Build-Tool mit extrem schnellem Hot-Module-Replacement. Die Entwicklungserfahrung ist deutlich besser als bei Webpack, und die Build-Zeiten sind kürzer.

Docker & Docker Compose - Begründung: Docker ermöglicht konsistente Entwicklungsumgebungen und einfaches Deployment. Docker Compose koordiniert Backend und Datenbank als Services, was die lokale Entwicklung vereinfacht und Production-Deployment vorbereitet.

5.2. Entwurf

5.2.1. Client-Server-Architektur mit REST-API

Die Anwendung folgt einer klassischen 3-Tier-Architektur:



5.2.2. Komponenten und Zusammenwirken

- **Frontend (Vue.js SPA)**
 - Rendert UI-Komponenten im Browser
 - Kommuniziert mit Backend über REST-API
 - Verwaltet Authentifizierungsstatus in Pinia Store
 - Nutzt Vue Router für Navigation
- **Backend (Deno + Oak v16)**

- Stellt REST-API-Endpunkte bereit
- Authentifiziert Requests via JWT-Middleware
- Validiert und verarbeitet Daten mit Oak's Context-basiertem Request-Handling
- Body-Parsing über `ctx.request.body` mit `bodyReader.type()` und `bodyReader.json()`
- Führt SQL-Queries auf Datenbank aus
- **Datenbank (MySQL)**
 - Speichert Benutzer, Transaktionen und Sparziele
 - Enforced Referential Integrity via Foreign Keys
 - Indiziert für Performance (`user_id`, `date`, `category`)

5.2.3. Entscheidungen

- **Stateless Backend:** JWT-Tokens ermöglichen horizontale Skalierung ohne Session-Storage
- **Oak Framework v16:** Native Deno-Integration mit Context-basiertem Request-Handling (`ctx.request`, `ctx.response`, `ctx.state`)
- **Body-Parser Middleware:** Custom Middleware für JSON-Parsing mit Oak v16's `bodyReader.type()` und `bodyReader.json()` Methoden
- **RESTful API:** Standardisierte HTTP-Methoden (GET, POST, PUT, DELETE) für klare API-Struktur
- **CORS-Konfiguration:** Explizite Whitelist für Frontend-Origins verhindert CSRF-Angriffe
- **Middleware-Pattern:** Authentifizierung als wiederverwendbare Oak-Middleware mit Context-basiertem Request-Handling

5.2.4. UI-Entwurf

Design-Entscheidungen:

- **Single-Page-Application (SPA)**
 - Keine Page-Reloads, flüssige Navigation
 - Router-basierte Navigation mit History-API
 - Authentifizierte Routen via Navigation Guards
- **Komponentenstruktur**
 - **Views:** Hauptseiten (LoginView, DashboardView, GoalTrackerView, TransactionFormView)
 - **Stores:** Globaler State (auth.js für Authentifizierung)
 - **Router:** Zentrale Routenkonfiguration mit Auth-Guards
- **User Experience**
 - **Loading States:** Spinner und Feedback während API-Calls
 - **Error Handling:** Benutzerfreundliche Fehlermeldungen
 - **Form Validation:** Client-seitige Validierung vor API-Calls
- **Styling**
 - **CSS-in-Component:** Scoped Styles in Vue-Komponenten
 - **Gradient-Design:** Moderne, visuell ansprechende UI
 - **Card-based Layout:** Klare visuelle Hierarchie

5.2.5. Datenmodell

Relationales Datenmodell mit 3 Haupttabellen:

users

|— id (PK)

|— email (UNIQUE)

|— password_hash
|— created_at
└— updated_at

transactions

|— transaction_id (PK)
|— user_id (FK → users.id)
|— type (ENUM: 'Income', 'Expense')
|— amount (DECIMAL)
|— category
|— date
|— description
|— created_at
└— updated_at

goals

|— goal_id (PK)
|— user_id (FK → users.id)
|— name
|— target_amount (DECIMAL)
|— current_savings (DECIMAL)
|— start_date
|— end_date
|— created_at
└— updated_at

5.2.6. Persistenzüberlegungen

- **Foreign Keys mit CASCADE:** Beim Löschen eines Benutzers werden automatisch alle Transaktionen und Sparziele gelöscht
- **Indizierung:**

- `idx_tx_user_date`: Schnelle Abfragen nach Benutzer und Datum
- `idx_tx_user_type`: Filterung nach Einnahmen/Ausgaben
- `idx_tx_user_category`: Kategorien-Analysen
- **DECIMAL für Geldbeträge**: Präzise Berechnungen ohne Floating-Point-Fehler
- **ENUM für Transaktionstypen**: Typsicherheit auf Datenbankebene
- **Timestamps**: Automatische `created_at` und `updated_at` für Audit-Trail

5.2.7. Interaktionsideen zwischen Client und Server

Funktionsabläufe:

- **Registrierung/Login**

Frontend → POST `/api/auth/register`

Backend → Hash Passwort (bcrypt)

Backend → Erstelle User in DB

Backend → Generiere JWT Token

Backend → Response mit Token

Frontend → Speichere Token in `localStorage`

Frontend → Setze Authorization Header

Frontend → Redirect zu Dashboard

- **Transaktion hinzufügen**

Frontend → POST `/api/transactions/expense` (JSON Body)

Backend → Body-Parser Middleware: `ctx.request.body` → `bodyReader.json()`

Backend → `ctx.state.body` enthält geparsete JSON-Daten

Backend → Middleware: `authenticateUser` validiert JWT Token

Backend → Extrahiere `user_id` aus `ctx.state.user.id`

Backend → Validiere `ctx.state.body` (Betrag, Kategorie, Datum)

Backend → INSERT INTO `transactions`

Backend → `ctx.response.status = 201, ctx.response.body = { message: ... }`

Frontend → Zeige Erfolgsmeldung

Frontend → Optional: Refresh Dashboard

- **Dashboard laden**

Frontend → GET `/api/transactions/summary?month=1&year=2024`

Backend → Middleware: `authenticateUser` validiert JWT Token

Backend → Extrahiere `user_id` aus `ctx.state.user.id`

Backend → URL-Parameter: `ctx.request.url.searchParams.get('month')`

Backend → SQL: SUM Income/Expense, GROUP BY category

Backend → `ctx.response.body = { currentSaldo, total_income, total_expense, categoryBreakdown }`

Frontend → Rendere Dashboard mit Daten

- **Geschützte Routen**

Frontend → Router Guard: `beforeEach()`

Frontend → Prüfe `authStore.isLoggedIn`

Frontend → Wenn nicht eingeloggt → Redirect zu `/login`

Frontend → Wenn eingeloggt → Erlaube Zugriff

Sicherheitsaspekte:

- **JWT in Authorization Header:** Token wird bei jedem Request mitgesendet
- **Token-Expiration:** Tokens laufen nach 1 Tag ab
- **CORS-Whitelist:** Nur erlaubte Origins können API aufrufen (via `ctx.response.headers.set()`)
- **Password Hashing:** Passwörter werden niemals im Klartext gespeichert
- **Context-basierte Datenvalidierung:** Alle Daten werden über `ctx.state.body` validiert
- **User-ID Prüfung:** Jeder Datenzugriff prüft `ctx.state.user.id` gegen Datenbank-Einträge

5.2.8. Datei-Übersicht

Backend

Datei	Zweck
backend/server.js	Hauptserver-Datei, Oak Application-Initialisierung, Middleware-Setup (CORS, Body-Parser, Error-Handler), Routing
backend/src/config/db.js	Datenbankverbindung (MySQL Connection Pool)
backend/src/controller/authControllers.js	Authentifizierungs-Logik (Register, Login, JWT-Generierung)
backend/src/routes/auth.js	Authentifizierungs-Routen (POST /register, POST /login)
backend/src/routes/transactions.js	Transaktions- und Sparziel-Routen (CRUD-Operationen, Betrag hinzufügen/entfernen)
backend/middleware/authenticate.js	JWT-Authentifizierungs-Middleware für geschützte Routen (Oak Context-basiert: ctx.request.headers, ctx.state.user)
backend/deno.json	Deno-Konfiguration, Tasks, npm-Import-Mappings
backend/public/index.html	Fallback-HTML für statische Dateien

Frontend

Datei	Zweck
frontend/src/main.js	Vue-App-Initialisierung, Pinia- und Router-Setup
frontend/src/App.vue	Root-Komponente, Header mit Logout, RouterView
frontend/src/router/index.js	Vue Router-Konfiguration, Routen, Navigation Guards
frontend/src/stores/auth.js	Pinia Store für Authentifizierung (Login, Register, Logout)
frontend/src/stores/counter.js	Beispiel-Pinia Store (kann entfernt werden)
frontend/src/views/LoginView.vue	Login/Registrierungs-Formular
frontend/src/views/DashboardView.vue	Haupt-Dashboard mit Saldo, Einnahmen, Ausgaben, Kategorien
frontend/src/views/TransactionForm.vue	Formular zum Hinzufügen/Bearbeiten von Transaktionen
frontend/src/views/GoalTracker.vue	Sparziel-Verwaltung (Anzeige, Erstellen, Bearbeiten, Betrag hinzufügen/entfernen)
frontend/src/views/CategoryChart.vue	Kategorien-Visualisierung (optional)
frontend/src/style.css	Globale Styles

frontend/vite.config.js	Vite-Konfiguration
frontend/package.json	npm-Dependencies und Scripts
frontend/index.html	HTML-Entry-Point

Datenbank & DevOps

Datei	Zweck
db/init.sql	Datenbankschema (CREATE TABLE Statements)
Dockerfile	Docker-Image für Backend (Deno-basiert)
docker-compose.yml	Orchestrierung von Backend und MySQL-Datenbank
docker-compose.dev.yml	Docker Compose für Entwicklung mit Hot Reload
start.sh	Start-Skript für Linux/macOS (Bash)
start.ps1	Start-Skript für Windows (PowerShell)
START.md	Kurzanleitung für den Start der Anwendung

Fremdcode-Dateien

Datei/Paket	Herkunft	Zweck
node_modules/ (Frontend)	npm	Vue.js, Vue Router, Pinia,

		Axios, Vite
node_modules/ (Backend)	Deno npm:	mysql2, bcryptjs, jsonwebtoken
deno.lock	Deno	Dependency-Lockfile für reproduzierbare Builds

6. Ergebnis des Projekts

6.1. Funktionalität

Sparo ist eine vollständig funktionsfähige Budget-Tracking-Anwendung mit folgenden Features:

- **Benutzerauthentifizierung**
 - Registrierung neuer Benutzer
 - Login mit E-Mail und Passwort
 - JWT-basierte Session-Verwaltung
 - Automatische Weiterleitung nach Login/Logout
- **Transaktionsverwaltung**
 - Einnahmen hinzufügen
 - Ausgaben hinzufügen
 - Transaktionen bearbeiten
 - Transaktionen löschen
 - Kategorisierung von Transaktionen
- **Dashboard**

- Monatliche Finanzübersicht
- Aktueller Saldo (Einnahmen - Ausgaben)
- Kategorien-Aufschlüsselung
- Visuelle Darstellung der Finanzdaten
- **Sparziel-Verwaltung**
 - Sparziele erstellen
 - Fortschritt verfolgen
 - Beträge zu Sparzielen hinzufügen
 - Beträge von Sparzielen entfernen
 - Monatliche Sparbeträge berechnen
 - Meilensteine (50%, 100%)

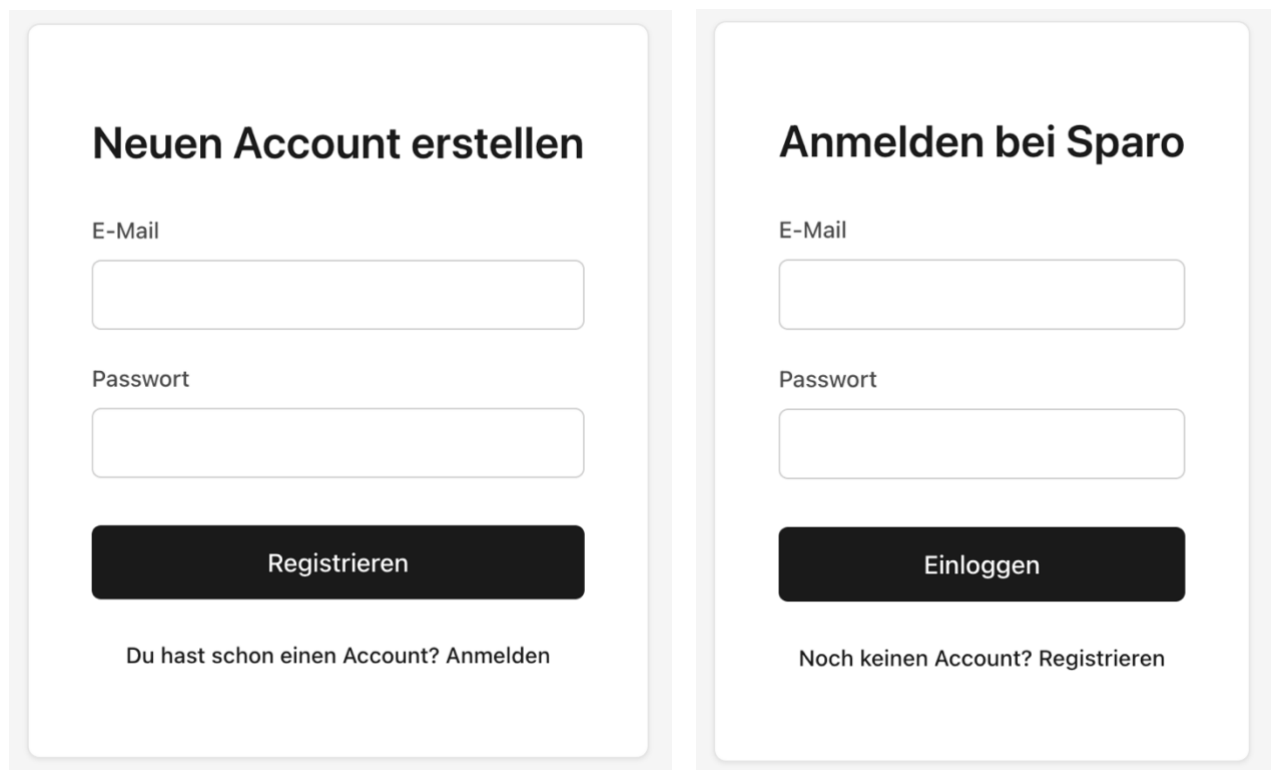
6.2. Screenshots

Screenshot 1: Login/Registrierungs-Seite

Die Login-Seite bietet zwei Modi:

- **Login:** Bestehende Benutzer können sich mit E-Mail und Passwort anmelden
- **Registrierung:** Neue Benutzer können einen Account erstellen

Funktionen: Die Authentifizierungsfunktion umfasst E-Mail- und Passwort-Eingabefelder sowie einen Toggle zum Umschalten zwischen Login- und Registrierungsmodus. Das System gewährleistet die automatische Weiterleitung zum Dashboard bei erfolgreicher Authentifizierung und zeigt bei ungültigen Daten entsprechende Fehlermeldungen an.



Neuen Account erstellen

E-Mail

Passwort

Registrieren

[Du hast schon einen Account? Anmelden](#)

Anmelden bei Sparo

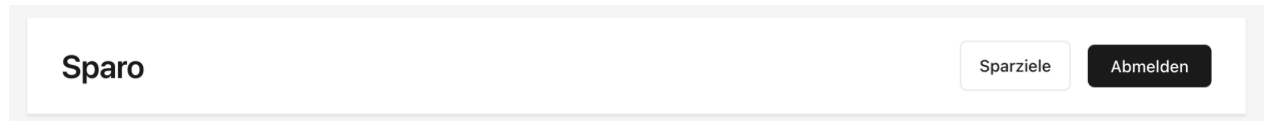
E-Mail

Passwort

Einloggen

[Noch keinen Account? Registrieren](#)

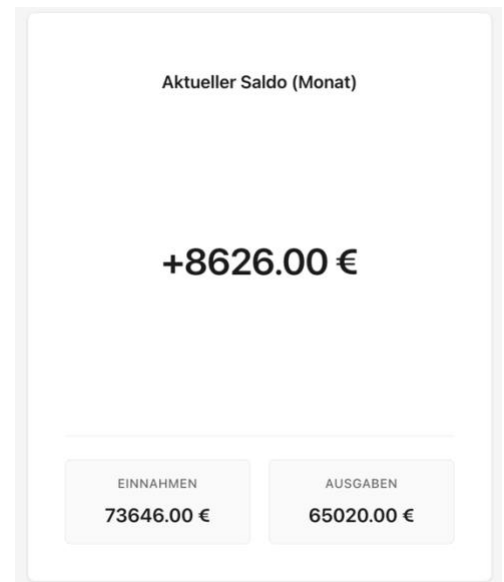
Screenshot 2: Navigation



Die Navigationsleiste der Anwendung wird durch das Logo „Sparo“ prominent eingeleitet und dient als zentrales Informationselement. Am rechten Ende der Leiste befinden sich die direkt zugänglichen Schaltflächen „Sparziele“ und „Abmelden“.

Screenshot 3: Aktueller Saldo

Der Ausschnitt visualisiert die monatliche Bilanz, bestehend aus der Zusammenfassung der Einnahmen und Ausgaben, sowie dem aktuellen Saldo.



Screenshot 4: Ausgaben nach Kategorie Chart

Der Ausschnitt visualisiert die Ausgaben nach Kategorien als Chart.



Screenshot 5: Sparziel Anzeige



Meine Sparziele

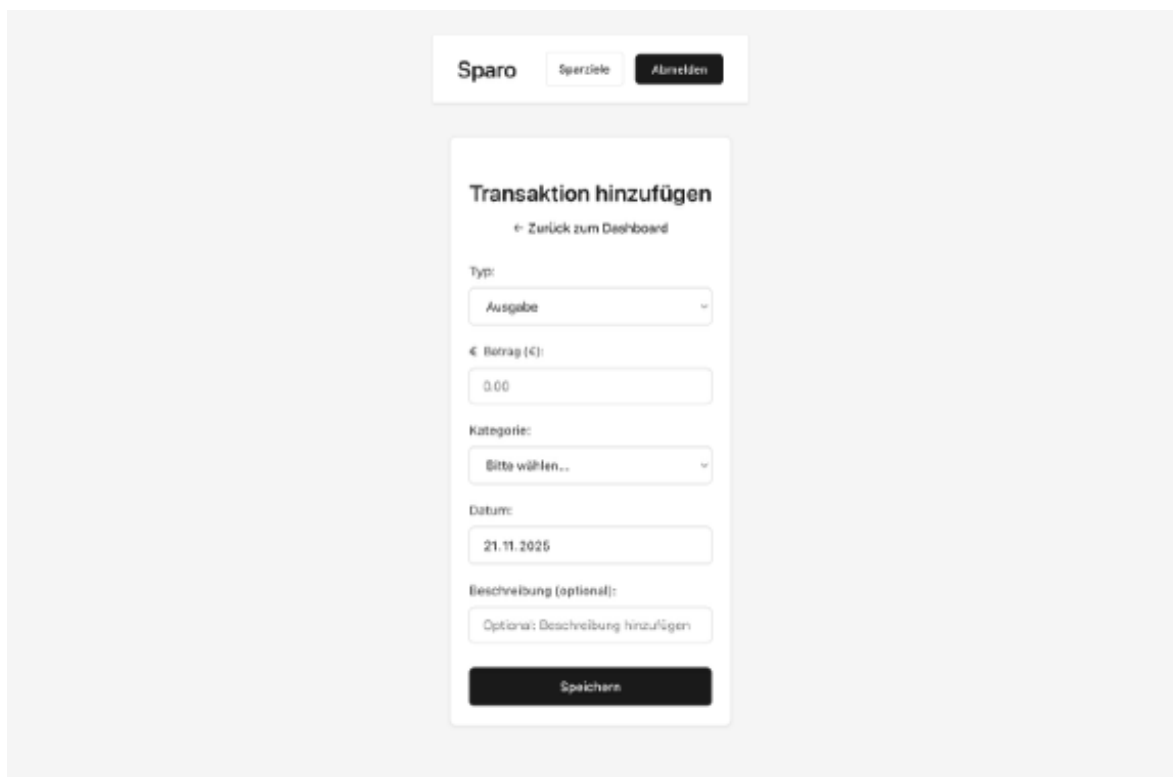
Urlaub Details →

0.00 € von 5000.00 €

0%

Der Bereich „Meine Sparziele“ zeigt in einem Card-Layout den Fortschritt spezifischer Sparziele, wie „Urlaub“. Dargestellt werden der aktuelle Betrag im Verhältnis zum Ziel (z.B. 0,00 € von 5000,00 €), ergänzt durch eine prozentuale Angabe und eine Fortschrittsleiste. Ein „Details →“-Link führt zu weiteren Informationen.

Screenshot 6: Transaktion hinzufügen



Sparo Sparziele Abschließen

Transaktion hinzufügen
← Zurück zum Dashboard

Typ:
Ausgabe

€ Betrag (€):
0.00

Kategorie:
Bitte wählen...

Datum:
21.11.2025

Beschreibung (optional):
Optional: Beschreibung hinzufügen

Speichern

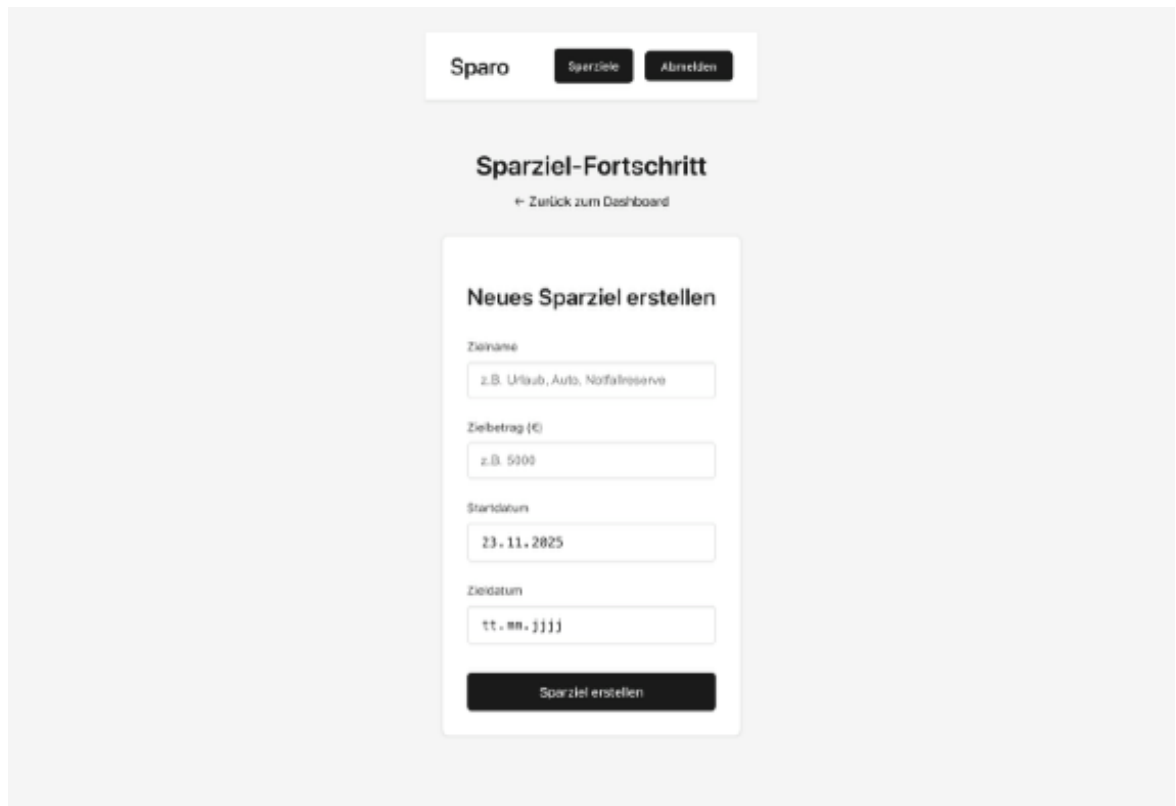
Das Transaktions-Formular ermöglicht das Erfassen neuer Einnahmen oder

Ausgaben:

Die Eingabemaske für Transaktionen ist klar strukturiert: Sie beginnt mit einer Dropdown-Auswahl zur Kennzeichnung als „Einnahme“ oder „Ausgabe“. Es folgt das Betragsfeld, das eine numerische Eingabe mit Validierung erfordert. Die Zuweisung erfolgt über ein Kategorie-Dropdown mit vordefinierten Optionen (wie Essen, Freizeit oder Gehalt). Das Datum wird per Auswahlfeld erfasst, wobei das heutige Datum voreingestellt ist, und ein optionales Textfeld steht für eine detaillierte Beschreibung zur Verfügung.

Die Benutzeroberfläche gewährleistet eine verbesserte Nutzererfahrung und Datenqualität durch Client-seitige Validierung, die bereits vor dem Absenden der Daten erfolgt. Nach dem Speichern erhält der Nutzer ein klares Feedback in Form von Erfolgs- oder Fehlermeldungen. Abschließend sorgt die automatische Formularbereinigung dafür, dass das Eingabefeld nach einem erfolgreichen Speichervorgang sofort zurückgesetzt und für die nächste Eingabe bereit ist.

Screenshot 7: Sparziel-Tracker



The screenshot displays the 'Sparziel-Fortschritt' page of the Sparziel-Tracker application. At the top, there is a navigation bar with the 'Sparo' logo and two buttons: 'Sparziele' and 'Abmelden'. Below the navigation bar, the page title 'Sparziel-Fortschritt' is shown, followed by a link '+ Zurück zum Dashboard'. The main content area features a form titled 'Neues Sparziel erstellen'. The form contains four input fields: 'Zielname' (with a placeholder 'z.B. Urlaub, Auto, Notfallreserve'), 'Zielbetrag (€)' (with a placeholder 'z.B. 5000'), 'Startdatum' (with a placeholder '23.11.2025'), and 'Zieldatum' (with a placeholder 'tt.mm.jjjj'). A 'Sparziel erstellen' button is located at the bottom of the form.

Die Sparziel-Verwaltung zeigt alle gesetzten Sparziele mit Fortschrittsanzeige:

Die Übersicht der Sparziele ist durch die Bezeichnung des Sparziels klar definiert und bietet eine visuelle Darstellung des Fortschritts in Form eines Fortschrittsbalkens mit Prozentangabe. Sie listet den Zielbetrag sowie den aktuellen Stand (bereits gesparte Summe) auf. Zudem werden automatisch der monatliche Sparbetrag (basierend auf der verbleibenden Zeit) und die verbleibenden Monate bis zum Zieldatum kalkuliert.

Die Verwaltung der Sparziele bietet volle Flexibilität: Nutzer können neue Sparziele erstellen, bestehende Ziele detailliert bearbeiten und nicht mehr benötigte Ziele einfach löschen. Zudem wird der Fortschritt durch die visuelle Hervorhebung wichtiger Meilensteine (z. B. das Erreichen von 50 % und 100 % des Zielbetrags) optisch unterstützt.

7. Reflexion

7.1. Herausforderungen

Herausforderung 1: Oak und Express

Problem: Ein anfängliches Problem war die fehlerhafte Interpretation der Aufgabenstellung, die zur Verwendung von Express anstelle des geforderten Oak-Frameworks führte. Dies machte eine vollständige Migration der gesamten Anwendung von Express auf Oak erforderlich.

Lösung: Durch YouTube Videos und KI- Unterstützung

Erkenntnis: Aufgabenstellung besser lesen!

Herausforderung 2: Oak v16 Body-Parsing

Problem: Oak v16 hat eine andere Body-Parsing-API als Express. `ctx.request.body` ist kein direktes Objekt, sondern ein `BodyReader` mit Methoden `type()` und `json()`.

Lösung: Zur Lösung des Body-Parsing-Problems wurde `ctx.request.body` als `BodyReader`-Objekt genutzt, wobei explizit `bodyReader.type()` und `bodyReader.json()` zur Verarbeitung aufgerufen wurden. Eine Custom Middleware setzt den geparsen Body im `ctx.state.body`.

Erkenntnis: Die wesentliche Erkenntnis ist, dass Oak v16 Promise-basierte Methoden verwendet und seine Context-basierte Architektur (`ctx.request/response/state`) einen konsistenteren Ansatz als das Express `req/res`-Pattern bietet.

Herausforderung 3: JWT-Authentifizierung

Problem: Verständnis, wie JWT-Tokens funktionieren und wie sie sicher im Frontend gespeichert werden.

Lösung: Die JWT-Lösung umfasste das Studium der Spezifikation, die Implementierung der Token-Generierung im Backend und die Speicherung in

localStorage (unter Beachtung der Risiken). Eine Middleware wurde zur Token-Validierung implementiert.

Erkenntnis: Die Erkenntnis ist, dass JWT eine skalierbare stateless Authentifizierung ermöglicht, die jedoch eine sorgfältige Implementierung erfordert.

Herausforderung 4: CORS-Fehler

Problem: Frontend (Port 5173) konnte nicht auf Backend (Port 3000) zugreifen aufgrund von CORS-Fehlern.

Lösung: Die Lösung war eine CORS-Middleware in Oak. Dazu wurden `ctx.response.headers.set`-Methoden verwendet, um `Access-Control-Allow-Origin` und `Access-Control-Allow-Credentials` zu konfigurieren und `OPTIONS`-Requests zu behandeln.

Erkenntnis: Die Erkenntnis ist, dass CORS ein notwendiges Sicherheitsfeature ist, das explizit konfiguriert werden muss. Oak nutzt dafür `ctx.response.headers` statt des Express `res.headers()`.

Herausforderung 5: Vue 3 Composition API

Problem: Umstellung von Options API (Vue 2) auf Composition API (Vue 3).

Lösung: Tutorials zur Composition API, Verwendung von `<script setup>` für kompakteren Code, Verständnis von `ref()`, `reactive()`, `computed()`

Erkenntnis: Composition API ist leistungsfähiger und ermöglicht bessere Code-Organisation.

Herausforderung 6: Datenbank-Performance

Problem: Langsame Queries bei vielen Transaktionen.

Lösung: Indizierung von `user_id`, `date`, `category`, Verwendung von `EXPLAIN` zur Query-Analyse, Connection Pooling für bessere Ressourcennutzung

Erkenntnis: Datenbank-Indizierung ist kritisch für Performance.

7.2. Unterstützung

Wer/Was hat geholfen?

- **Offizielle Dokumentationen**
 - Deno-Dokumentation für Runtime-Features
 - Vue 3-Dokumentation für Composition API
 - Express-Dokumentation für Middleware
- **Stack Overflow**
 - Spezifische Fehlermeldungen googeln
 - Best Practices für JWT-Implementierung
 - CORS-Konfigurations-Beispiele
- **YouTube-Tutorials**
 - Vue 3 Composition API Crash Course
 - JWT-Authentifizierung Tutorials
 - Docker für Anfänger
- **GitHub-Repositories**
 - Beispiel-Projekte für REST-APIs
 - Best Practices für Vue 3-Projekte
- **KI-Tools**
 - Code-Erklärungen
 - Debugging-Hilfe
 - Architektur-Beratung
 - Deno Implementierung
 - Wechsel von Express auf Oak

- Styling
- Datei Erstellung/Hilfe: start.sh, start.ps1, START.md, server.js, init.sql
- **Community**
 - Deno Community Forum

7.3. Lernerfolge / Fazit

7.3.1. Was habe ich gelernt?

- **Deno Runtime & Oak v16**
 - Moderne JavaScript/TypeScript-Runtime
 - Sicherheits-Features durch explizite Permissions
 - Native npm-Paket-Unterstützung
 - Oak v16 Context-basierte Middleware-Architektur
 - Modernes Body-Parsing mit `bodyReader.type()` und `bodyReader.json()`
- **REST-API-Design mit Oak v16**
 - RESTful-Endpunkt-Struktur mit Oak Router
 - HTTP-Status-Codes über `ctx.response.status` setzen
 - Request/Response-Formatierung über `ctx.request` und `ctx.response`
 - Context-basierte Datenübergabe via `ctx.state`
- **JWT-Authentifizierung mit Oak**
 - Token-basierte Authentifizierung
 - Middleware-Pattern für geschützte Routen (Oak Context-basiert)
 - Token-Expiration und Refresh-Strategien

- `ctx.request.headers.get('authorization')` für Header-Zugriff
 - `ctx.state.user` für Benutzerdaten in nachfolgenden Middlewares
- **Vue 3 Composition API**
 - `<script setup>` Syntax
 - Reaktive Daten mit `ref()` und `reactive()`
 - Computed Properties und Watchers
- **Pinia State Management**
 - Store-Struktur für globale Daten
 - Actions für asynchrone Operationen
 - Getters für abgeleitete Daten
- **Docker & Docker Compose**
 - Containerisierung von Anwendungen
 - Service-Orchestrierung
 - Volume-Management für Datenbanken
- **Datenbank-Optimierung**
 - Indizierung für Performance
 - Foreign Keys für Datenintegrität
 - Query-Optimierung

7.3.2. Wie kann ich das Wissen zukünftig einsetzen?

Das Projekt schärft Full-Stack-Kenntnisse und das REST-API-Design für kommende Projekte. Die Implementierung der JWT-Authentifizierung ist direkt übertragbar. Containerisierung mittels Docker verbessert Entwicklung und Deployment. Die erworbenen Kenntnisse in Vue.js-Entwicklung erleichtern zukünftige Vue 3-Projekte.

7.3.3. Was lief gut?

- Klare Projektstruktur von Anfang an
- Schrittweise Entwicklung (Backend → Frontend → Integration)
- Verwendung moderner Technologien (Deno, Vue 3)
- Vollständige Funktionalität aller geplanten Features
- Docker-Containerisierung für einfaches Deployment

7.3.4. Was würde ich verbessern?

- TypeScript für bessere Typsicherheit verwenden
- Zentralisierte Fehlerbehandlung im Backend
- Mehr visuelles Feedback
- Mehr Funktionen

7.3.5. Was kann man nun mit der Anwendung machen?

- **Erweiterte Features**
 - Wiederkehrende Transaktionen (Monatlich, Jährlich)
 - Budget-Limits pro Kategorie
 - Export-Funktion (CSV, PDF)
 - Multi-Währung-Unterstützung
 - Kategorien-Verwaltung (Benutzerdefinierte Kategorien)
- **Mobile App**
 - React Native oder Flutter App
 - Push-Notifications für Budget-Warnungen
 - Offline-Modus mit Synchronisation
- **Analytics & Reporting**

- Jahresübersichten
- Trend-Analysen
- Vorhersagen basierend auf historischen Daten
- Visualisierungen (Charts, Graphs)
- **Kollaboration**
 - Gemeinsame Budgets (Familie, WG)
 - Freigabe-Funktionen
 - Kommentare zu Transaktionen
- **Integrationen**
 - Bank-API-Integration (automatische Transaktions-Import)
 - E-Mail-Benachrichtigungen
 - Kalender-Integration

7.3.6. Wie kann sie weiterentwickelt werden?

- TypeScript-Migration
- Verbesserte Fehlerbehandlung
- API-Dokumentation (Swagger/OpenAPI)
- GraphQL-API als Alternative zu REST
- Machine Learning für Budget-Vorhersagen

7.4. KI-Einsatz

7.4.1. Verwendete KI-Systeme

- **Cursor AI**
 - **Zweck:** Code-Generierung, Debugging-Hilfe, Architektur-Beratung

- **Einsatzgebiete:**
 - Umstellung von Express auf Oak v16
 - Oak v16 Body-Parsing-Implementierung (bodyReader.type(), bodyReader.json())
 - Context-basierte Middleware-Architektur
 - Erstellung von REST-API-Endpunkten mit Oak Router
 - Vue 3 Component-Struktur
 - Fehlerbehebung bei CORS-Problemen
 - Datenbank-Schema-Optimierung
 - Deno Hilfe
- **ChatGPT**
 - **Zweck:** Konzeptionelle Fragen, Best Practices, Code-Erklärungen
 - **Einsatzgebiete:**
 - JWT-Authentifizierung verstehen
 - Vue 3 Composition API lernen
 - REST-API-Design-Patterns
 - Datenbank-Indizierungs-Strategien

7.4.2. Wie wurde KI eingesetzt?

- **Code-Generierung:** Schnellere Implementierung von Boilerplate-Code
- **Debugging:** Erklärung von Fehlermeldungen und Lösungsvorschläge
- **Lernen:** Erklärung komplexer Konzepte (JWT, CORS, etc.)
- **Architektur:** Beratung bei Design-Entscheidungen

7.4.3. Transparenz und Verantwortung

Alle KI-generierten Code-Abschnitte wurden überprüft, angepasst und verstanden. Die KI diene als Hilfsmittel, nicht als Ersatz für eigenständiges Lernen und Verstehen.

Verantwortung: Der Autor übernimmt die volle Verantwortung für den Code. Alle Fehler werden dem Autor zugerechnet, unabhängig davon, ob KI-Hilfsmittel verwendet wurden.

A. Installationsanleitung

Voraussetzungen

- Docker Desktop installiert und gestartet (muss vor dem Start laufen!) –
- Docker Compose installiert (meist mit Docker Desktop enthalten)
- Node.js (Version 20 oder höher) und npm installiert (für Frontend)
- Deno installiert (für Backend im lokalen Modus)
- Git (zum Klonen des Repositories, optional)

Schnellstart mit Docker (Empfohlen)

Die einfachste Methode, die Anwendung zu starten:

1. Projekt herunterladen

Bevor Sie die Anwendung starten können, müssen Sie das Projekt zuerst herunterladen:

Option 1: Git Repository klonen

```
git clone https://github.com/elischultzeon24/Sparo\_App.git
```

```
cd Sparo
```

Option 2: ZIP-Datei entpacken

Linux/macOS

```
unzip Sparo.zip
```

```
cd Sparo
```


Windows (PowerShell)

Expand-Archive -Path Sparo.zip -DestinationPath .

cd Sparo

2. Sparo starten

Linux/macOS:

Im Projektroot-Verzeichnis

./start.sh docker

Windows (PowerShell):

Im Projektroot-Verzeichnis

.\start.ps1 docker

Dies startet:

- Backend-Server (Docker) auf Port 3000
- Datenbank (Docker) auf Port 3306

Danach Frontend separat starten:

cd frontend

npm install

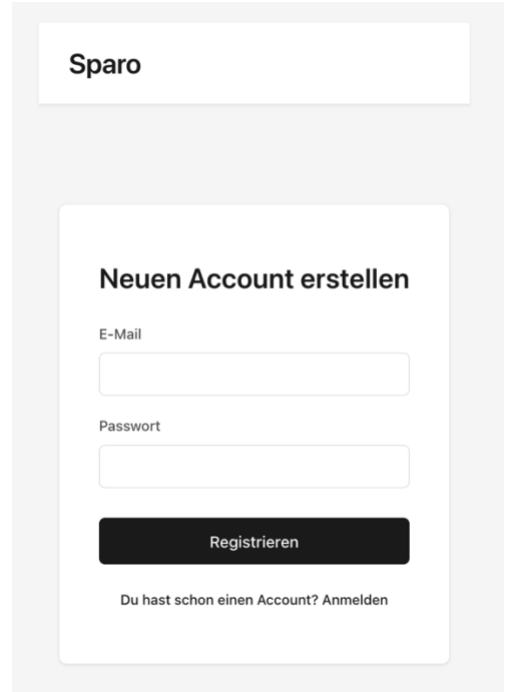
npm run dev

Das Frontend läuft dann auf <http://localhost:5173>

B. Benutzerdokumentation

Szenario 1: Neuen Account erstellen

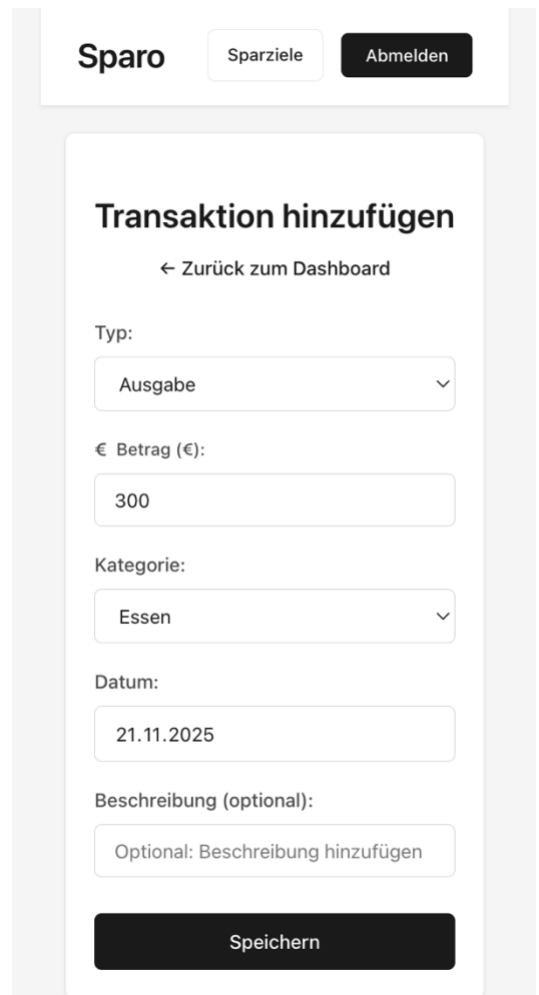
- Öffne <http://localhost:5173> im Browser
- Klicke auf “Registrieren” oder navigiere zu `/register`
- Gib E-Mail und Passwort ein
- Klicke auf “Registrieren”
- Du wirst automatisch zum Dashboard weitergeleitet



The screenshot shows the registration page of the Sparo application. At the top, there is a header with the word "Sparo". Below it, the main heading is "Neuen Account erstellen". There are two input fields: "E-Mail" and "Passwort". Below these fields is a dark button labeled "Registrieren". At the bottom, there is a link that says "Du hast schon einen Account? Anmelden".

Szenario 2: Transaktion hinzufügen

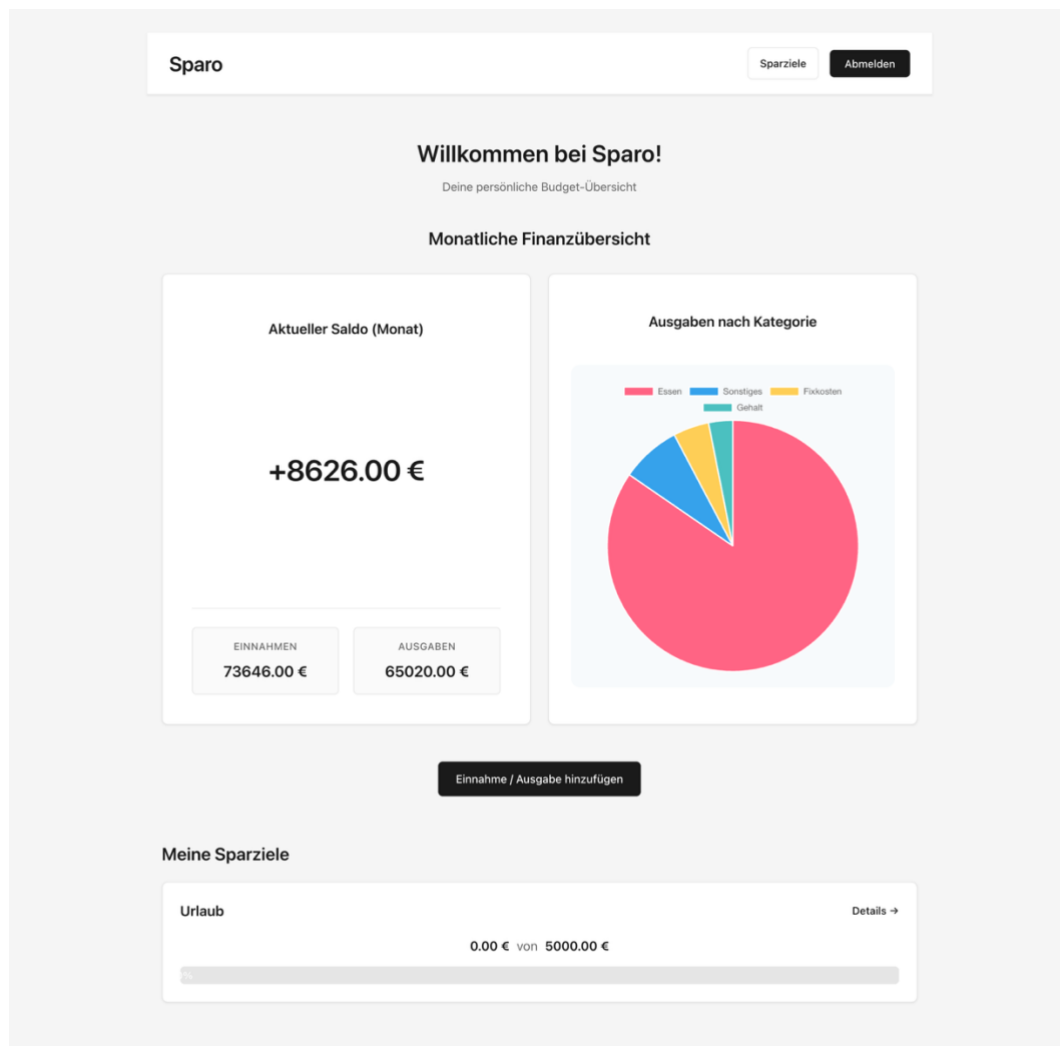
- Logge dich ein
- Klicke auf “Transaktion hinzufügen” (oder navigiere zu `/add`)
- Wähle Typ: “Einnahme” oder “Ausgabe”
- Gib Betrag, Kategorie, Datum und optional Beschreibung ein
- Klicke auf “Speichern”
- Die Transaktion erscheint im Dashboard



The screenshot shows the "Transaktion hinzufügen" form in the Sparo application. At the top, there is a header with the word "Sparo" and two buttons: "Sparziele" and "Abmelden". Below the header, the main heading is "Transaktion hinzufügen". There is a link that says "← Zurück zum Dashboard". There are four input fields: "Typ:" (with a dropdown menu showing "Ausgabe"), "€ Betrag (€):" (with the value "300"), "Kategorie:" (with a dropdown menu showing "Essen"), and "Datum:" (with the value "21.11.2025"). There is also an optional field for "Beschreibung (optional):" with the placeholder text "Optional: Beschreibung hinzufügen". At the bottom, there is a dark button labeled "Speichern".

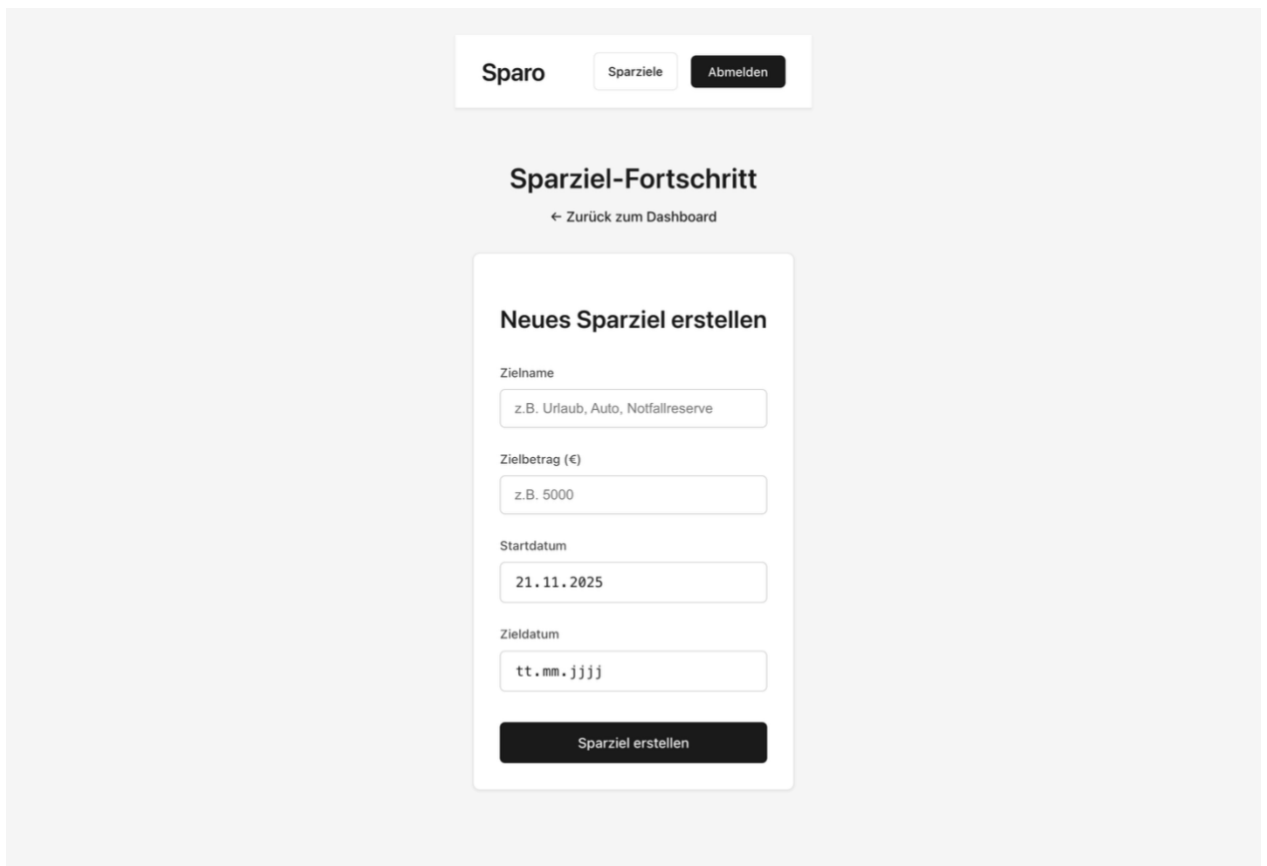
Szenario 3: Dashboard anzeigen

Nach erfolgreichem Login gelangst du automatisch auf das Dashboard, welches als zentrale Übersichtsseite dient. Das Dashboard liefert sofort alle relevanten Finanzinformationen des aktuellen Monats. Konkret werden vier Schlüsselkennzahlen angezeigt: der Aktuelle Saldo (berechnet aus Einnahmen minus Ausgaben des laufenden Monats), die Gesamteinnahmen des Monats, die Gesamtausgaben des Monats sowie eine detaillierte Kategorien-Übersicht, die eine Aufschlüsselung der Ausgaben nach ihren jeweiligen Kategorien ermöglicht.



Szenario 4: Sparziel erstellen

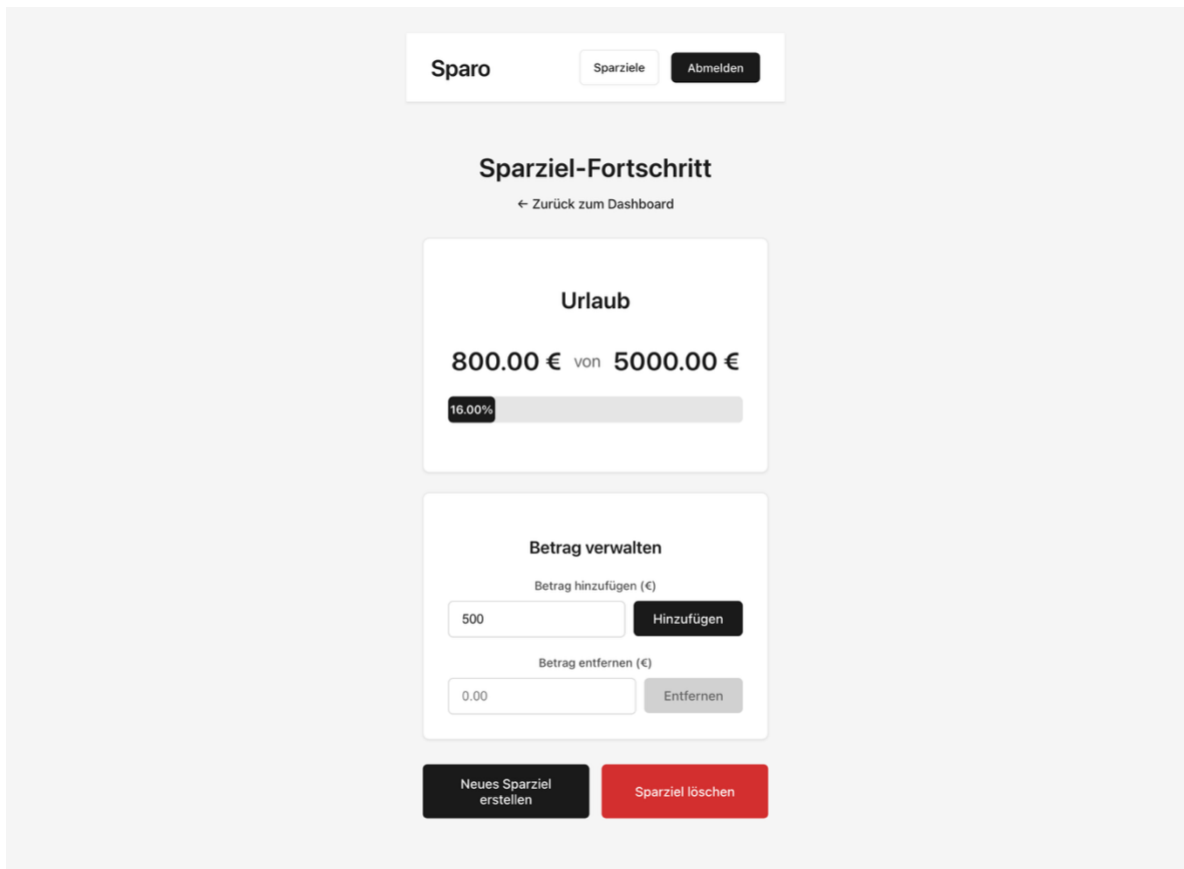
Die Erstellung eines neuen Sparziels erfolgt in wenigen Schritten: Zunächst navigierst du zum Bereich „Sparziele“ (alternativ über die Route /goals). Dort klickst du auf die Schaltfläche „Neues Sparziel“. Es öffnet sich ein Formular, in das der Nutzer den Namen, den Zielbetrag sowie das Start- und Enddatum für das Sparvorhaben eingibt. Nach dem Klick auf „Erstellen“ wird das neue Sparziel in der Übersicht angezeigt. Die Darstellung beinhaltet einen Fortschrittsbalken, der den bisherigen Fortschritt visualisiert, den berechneten monatlichen Sparbetrag, der notwendig ist, um das Ziel fristgerecht zu erreichen, sowie die verbleibenden Monate bis zum Zieltermin.



The screenshot displays a web interface for managing savings goals. At the top, there is a navigation bar with the 'Sparo' logo and two buttons: 'Sparziele' and 'Abmelden'. Below this, the main heading is 'Sparziel-Fortschritt', with a link '← Zurück zum Dashboard'. The central focus is a form titled 'Neues Sparziel erstellen'. This form contains four input fields: 'Zielname' with a placeholder 'z.B. Urlaub, Auto, Notfallreserve', 'Zielbetrag (€)' with a placeholder 'z.B. 5000', 'Startdatum' with a placeholder '21.11.2025', and 'Zieldatum' with a placeholder 'tt.mm.jjjj'. A dark button labeled 'Sparziel erstellen' is positioned at the bottom of the form.

Szenario 5: Betrag zu Sparziel hinzufügen/entfernen

Um deine Sparziele zu verwalten, navigierst du entweder direkt zur Übersichtsseite (oder zur Route /goals) und wählst dort das Ziel aus, das du bearbeiten möchtest. Im Bereich „Betrag verwalten“ kannst du nun Ein- oder Auszahlungen vornehmen: Gib einen Betrag ein und klicke auf „Hinzufügen“, um Geld einzuzahlen, oder klicke auf „Entfernen“, um eine Entnahme zu buchen. Beachte, dass sich der Fortschrittsbalken automatisch aktualisiert, um dir deinen aktuellen Status zu zeigen. Erreichst du einen wichtigen Meilenstein – nämlich 50 % oder 100 % des Zielbetrags – wird dir ein motivierender Meilenstein-Badge angezeigt.



The screenshot shows a web interface for managing savings goals. At the top, there's a header with the word "Sparo" and two buttons: "Sparziele" and "Abmelden". Below this is the main title "Sparziel-Fortschritt" with a link "← Zurück zum Dashboard".

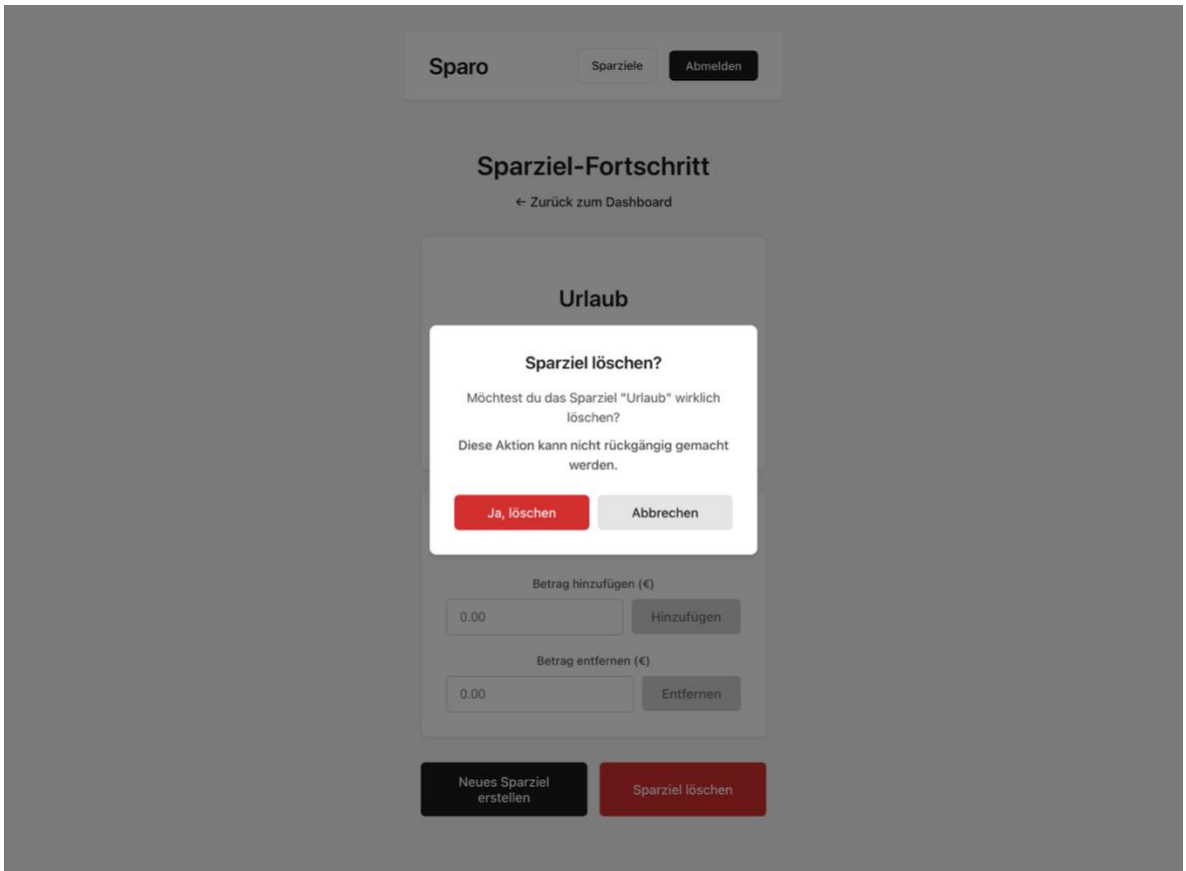
The main content area is divided into two sections:

- Urlaub**: A card showing a progress bar. The current amount is 800.00 € and the target is 5000.00 €. The progress bar is at 16.00%.
- Betrag verwalten**: A card with two input fields and buttons. The first field is labeled "Betrag hinzufügen (€)" and contains the value 500, with a "Hinzufügen" button next to it. The second field is labeled "Betrag entfernen (€)" and contains the value 0.00, with an "Entfernen" button next to it.

At the bottom, there are two buttons: "Neues Sparziel erstellen" and "Sparziel löschen".

Szenario 6: Sparziel löschen

Um ein Sparziel zu entfernen, navigierst du zunächst zum jeweiligen Sparziel. Dort klickst du auf „Löschen“ und bestätigst anschließend den Löschvorgang, um das Ziel endgültig zu entfernen.



The screenshot shows a web interface for managing savings goals. At the top, there's a navigation bar with 'Sparo', 'Sparziele', and 'Abmelden'. The main heading is 'Sparziel-Fortschritt' with a link to '← Zurück zum Dashboard'. Below this, a card titled 'Urlaub' contains a confirmation dialog box. The dialog asks 'Sparziel löschen?' and 'Möchtest du das Sparziel "Urlaub" wirklich löschen?'. It also states 'Diese Aktion kann nicht rückgängig gemacht werden.' and provides two buttons: 'Ja, löschen' (red) and 'Abbrechen' (grey). Below the dialog, there are input fields for 'Betrag hinzufügen (€)' and 'Betrag entfernen (€)', both currently showing '0.00', with corresponding 'Hinzufügen' and 'Entfernen' buttons. At the bottom, there are two buttons: 'Neues Sparziel erstellen' (dark grey) and 'Sparziel löschen' (red).

Literaturverzeichnis

Deno Foundation (2025) Deno: A secure runtime for JavaScript and TypeScript. Verfügbar unter: <https://deno.land/manual> (Zugriff am 9. November 2025).

Oak Framework (2025) Oak v16: Middleware framework for Deno. Verfügbar unter: <https://deno.land/x/oak> (Zugriff am 20. November 2025).

Vue.js (2025) Vue.js 3: The Progressive JavaScript Framework. Verfügbar unter: <https://vuejs.org/guide/introduction.html> (Zugriff am 9. November 2025).

MySQL (2025) MySQL 8.0 Reference Manual. Verfügbar unter: <https://dev.mysql.com/doc/refman/8.0/en/> (Zugriff am 9. November 2025).

Jones, M. (2020) JSON Web Token (JWT) Introduction. Verfügbar unter: <https://jwt.io/introduction/> (Zugriff am 9. November 2025).

bcryptjs (2025) bcryptjs: Optimized bcrypt in JavaScript. Verfügbar unter: <https://www.npmjs.com/package/bcryptjs> (Zugriff am 10. November 2025).

Docker Inc. (2025) Docker Documentation. Verfügbar unter: <https://docs.docker.com/> (Zugriff am 12. November 2025).

Axios (2025) Axios HTTP Client Documentation. Verfügbar unter: <https://axios-http.com/docs/intro> (Zugriff am 9. November 2025).

OpenAI (2023) ChatGPT: Language Model API. Verfügbar unter: <https://openai.com/api/> (Zugriff am 23. November 2025).

Cursor AI (2025) Cursor AI for Code Generation and Debugging. Verfügbar unter: <https://cursor.so> (Zugriff am 16. November 2025).

Stack Overflow (2025) Community-driven Q&A for programming. Verfügbar unter: <https://stackoverflow.com/> (Zugriff am 10. November 2025).

Nx engineers (2023) Converting an Express API To Deno. YouTube Video, 26. Februar. Verfügbar unter: <https://www.youtube.com/watch?v=Um8xXR54upQ> (Zugriff am 21. November 2025).

Stack Overflow (2020) Route router to another router in Oak framework like Express.js. Verfügbar unter: <https://stackoverflow.com/questions/62932706/route-router-to-another-router-in-oak-framework-like-expressjs> (Zugriff am 21. November 2025).

Oak Server (2018) oakserver/oak: A middleware framework for handling HTTP connections in Deno. GitHub Repository. Verfügbar unter: <https://github.com/oakserver/oak> (Zugriff am 21. November 2025).

Selbstständigkeitserklärung und Erklärung zum Nachweis der Verwendung von KI-Systemen bei Erstellung einer wissenschaftlichen Arbeit

(einzufügen nach allen Verzeichnissen der Arbeit)

Schultze, Elias

3417830

Nachname, Vorname
Matrikelnummer

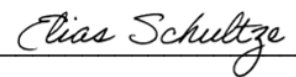
Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: (...) selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ich erkläre zudem, dass ich *(Zutreffendes ankreuzen)*

- ☐ Generative Systeme (Text-, Bild-, Codeerstellung etc.), wie beispielsweise den textbasierten Chatbot ChatGPT von OpenAI, bei Erstellung der vorliegenden wissenschaftlichen Arbeit **nicht** verwendet habe.
- ☒ Generative Systeme (Text-, Bild-, Codeerstellung etc.) bei Erstellung der vorliegenden wissenschaftlichen Arbeit in der nachfolgend beschriebenen Art und dem nachfolgend beschriebenen Umfang verwendet habe (Bitte in der nachfolgenden Tabelle näher konkretisieren; Hinweis: bei Unsicherheiten, ob und wie ein verwendetes KI-System anzugeben ist, stimmen Sie sich mit Ihrer/m Betreuer/in ab).

Waldbrunn, 23. November 2025

Ort, Datum



Unterschrift der/des Studierenden

Genutzter Funktionsumfang im Hinblick auf

- **Themenerfassung und Strukturierung** (Aufgabenstellung oder Präzisierung, Forschungsansatz, Gliederung)

Produktname des eingesetzten Hilfsmittels	Link, URL	Genutzter Funktionsumfang/Beschreibung der Verwendungsweise
ChatGPT	https://chatgpt.com/	Beratung zu Best Practices und konzeptionellen Fragen bezüglich REST-API-Design-Patterns und Datenbank-Indizierungs-Strategien.
Cursor AI	https://cursor.com/	Beratung bei Architektur-Design-Entscheidungen und für die Context-basierte Middleware-Architektur.

- **Themenbearbeitung** (Darstellung/Beschreibung des Problems, Darlegung von wissenschaftlichen Grundlagen, Schlussfolgerungen allgemein und für das konkrete Thema/Erkenntnisgewinn, Evaluierung des Textes/Feedback der IT/KI)

Produktname des eingesetzten Hilfsmittels	Link, URL	Genutzter Funktionsumfang/Beschreibung der Verwendungsweise
ChatGPT	https://chatgpt.com/	Erklärung komplexer Konzepte und des Verständnisses von JWT-Authentifizierung und CORS-Problemen. Lernen der Vue 3 Composition API.
Cursor AI	https://cursor.com/	Code-Generierung zur schnelleren Implementierung von Boilerplate-Code. Konkrete Umsetzung: Umstellung von Express auf Oak v16, Oak v16 Body-Parsing-Implementierung (bodyReader.type(), bodyReader.json()), Erstellung von REST-API-Endpunkten mit Oak Router, und Vue 3 Component-Struktur.
Cursor AI	https://cursor.com/	Umstellung von Express auf Oak v16 Oak v16 Body-Parsing-Implementierung (bodyReader.type(), bodyReader.json()) Context-basierte Middleware-Architektur Erstellung von REST-API-Endpunkten mit Oak Router Vue 3 Component-Struktur Fehlerbehebung bei CORS-Problemen Datenbank-Schema-Optimierung Deno Hilfe Datei Erstellung/Hilfe: start.sh, start.ps1,

		START.md, server.js, init.sql
--	--	-------------------------------

- **Quellenrecherche, -auswahl und -auswertung** (Welche Quellen wurden durch das IT-/KI-Tool gefunden, wie erfolgte die weitere Recherche)

Produktname des eingesetzten Hilfsmittels	Link, URL	Genutzter Funktionsumfang/Beschreibung der Verwendungsweise
ChatGPT	https://chatgpt.com/	Keine direkte Quellenrecherche, sondern Klärung von konzeptionellen Fragen und Best Practices als Grundlage für die Literaturrecherche.
Cursor AI	https://cursor.com/	Deno Hilfe (Erklärung von Deno-Modulen/Funktionen) und Datenbank-Schema-Optimierung.
Perplexity	https://www.perplexity.ai/	Quellen Recherche

- **Formale Gestaltung, insbesondere Sprache** (Welche Eingabe wurde an die IT/KI getätigt? Textgenerierung, Textkorrektur, Paraphrasieren und Umschreiben, Übersetzen, kreatives Schreiben)

Produktname des eingesetzten Hilfsmittels	Link, URL	Genutzter Funktionsumfang/Beschreibung der Verwendungsweise
ChatGPT	https://chatgpt.com/	Rechtschreibung Grundaufbau Dokumentation

Hinweis:

Geben Sie in der jeweiligen Rubrik die Kapitel in Ihrer wissenschaftlichen Arbeit an und erläutern Sie die Art und Weise sowie den Umfang der von Ihnen genutzten Generativen Systeme (Text, Bild, Code etc.).